

PLSQL EXERCISE:-

Schema Creation:

Step 1: Create the table

```
1 CREATE TABLE Customers (  
2     CustomerID NUMBER PRIMARY KEY,  
3     Name VARCHAR2(100),  
4     DOB DATE,  
5     Balance NUMBER,  
6     LastModified DATE  
7 );  
8  
9 CREATE TABLE Accounts (  
10     AccountID NUMBER PRIMARY KEY,  
11     CustomerID NUMBER,  
12     AccountType VARCHAR2(20),  
13     Balance NUMBER,  
14     LastModified DATE,  
15     FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)  
16 );  
17  
18 CREATE TABLE Transactions (  
19     TransactionID NUMBER PRIMARY KEY,  
20     AccountID NUMBER,  
21     TransactionDate DATE,  
22     Amount NUMBER,  
23     TransactionType VARCHAR2(10),  
24     FOREIGN KEY (AccountID) REFERENCES Accounts (AccountID)  
25 );  
26  
27 CREATE TABLE Loans (  
28     LoanID NUMBER PRIMARY KEY,  
29     CustomerID NUMBER,  
30     LoanAmount NUMBER,  
31     InterestRate NUMBER,  
32     StartDate DATE,  
33     EndDate DATE,  
34     FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)  
35 );  
36  
37 FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)  
38 );  
39  
40 CREATE TABLE Employees (  
41     EmployeeID NUMBER PRIMARY KEY,  
42     Name VARCHAR2(100),  
43     Position VARCHAR2(50),  
44     Salary NUMBER,  
45     Department VARCHAR2(50),  
46     HireDate DATE  
47 );
```

Step 2: Add the missing column

```
46  
47 ALTER TABLE Customers  
48 ADD IsVIP CHAR(1) DEFAULT 'N';
```

Table CUSTOMERS altered.
Elapsed: 00:00:00.025

Step 3: Create AuditLog Table

```
51 CREATE TABLE AuditLog(  
52     LogID NUMBER GENERATED BY DEFAULT AS IDENTITY,  
53     TransactionID NUMBER,  
54     LogDate DATE,  
55     Action VARCHAR2(50)  
56 );
```

Table AUDITLOG created.

Elapsed: 00:00:00.016

Step 4: Insert Sample Data

```
58 INSERT INTO Customers VALUES  
59 (1, 'John Doe', TO_DATE('1985-05-15', 'YYYY-MM-DD'), 1000, SYSDATE, 'N');  
60  
61 INSERT INTO Customers VALUES  
62 (2, 'Jane Smith', TO_DATE('1990-07-20', 'YYYY-MM-DD'), 1500, SYSDATE, 'N');  
63  
64 INSERT INTO Accounts VALUES  
65 (1, 1, 'Savings', 1000, SYSDATE);  
66  
67 INSERT INTO Accounts VALUES  
68 (2, 2, 'Checking', 1500, SYSDATE);  
69  
70 INSERT INTO Transactions VALUES  
71 (1, 1, SYSDATE, 200, 'Deposit');  
72  
73 INSERT INTO Transactions VALUES  
74 (2, 2, SYSDATE, 300, 'Withdrawal');  
75  
76 INSERT INTO Loans VALUES  
77 (1, 1, 5000, 5, SYSDATE, ADD_MONTHS(SYSDATE, 60));  
78  
79 INSERT INTO Employees VALUES  
80 (1, 'Alice Johnson', 'Manager', 70000, 'HR',  
81 TO_DATE('2015-06-15', 'YYYY-MM-DD'));  
82  
83 INSERT INTO Employees VALUES  
84 (2, 'Bob Brown', 'Developer', 60000, 'IT',  
85 TO_DATE('2017-03-20', 'YYYY-MM-DD'));  
86  
87 COMMIT;
```

```
1 row inserted.  
Elapsed: 00:00:00.002  
  
SQL> COMMIT  
  
Commit complete.  
Elapsed: 00:00:00.002
```

Exercise 1: Control Statement

- **Scenario 1:** The bank wants to apply a discount to loan interest rates for customers above 60 years old.

Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Code:

```
90
91 BEGIN
92     FOR rec IN (
93         SELECT l.LoanID
94         FROM Loans l
95         JOIN Customers c
96         ON l.CustomerID=c.CustomerID
97         WHERE FLOOR(MONTHS_BETWEEN(SYSDATE,c.DOB)/12)>60
98     )
99     LOOP
100         UPDATE Loans
101         SET InterestRate=InterestRate-1
102         WHERE LoanID=rec.LoanID;
103     END LOOP;
104
105     COMMIT;
106 END;
107
```

Output:

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.158

- **Scenario 2:** A customer can be promoted to VIP status based on their balance.

Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Code:

```
108
109 BEGIN
110     FOR rec IN
111     (
112         SELECT CustomerID
113         FROM Customers
114         WHERE Balance>10000
115     )
116     LOOP
117         UPDATE Customers
118         SET IsVIP='Y'
119         WHERE CustomerID=rec.CustomerID;
120     END LOOP;
121
122     COMMIT;
123 END;
124
```

Output:

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.157

- **Scenario 3:** The bank wants to send reminders to customers whose loans are due within the next 30 days.

Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Code:

```
126 BEGIN
127
128 FOR rec IN
129 (
130 SELECT c.Name,
131        l.LoanID,
132        l.EndDate
133 FROM Customers c
134 JOIN Loans l
135 ON c.CustomerID=l.CustomerID
136 WHERE l.EndDate
137 BETWEEN SYSDATE
138 AND SYSDATE+30
139 )
140
141 LOOP
142
143 DBMS_OUTPUT.PUT_LINE (
144 'Reminder sent to '
145 || rec.Name);
146
147 END LOOP;
148
149 END;
```

Output:

```
PL/SQL procedure successfully completed.

Elapsed: 00:00:00.020
```

Exercise 2: Error Handling

- **Scenario 1:** Handle exceptions during fund transfers between accounts.

Question: Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

Code:

```

151
152 CREATE OR REPLACE PROCEDURE SafeTransferFunds (
153     p_from NUMBER,
154     p_to NUMBER,
155     p_amount NUMBER
156 )
157 IS
158     v_balance NUMBER;
159 BEGIN
160     SELECT Balance
161     INTO v_balance
162     FROM Accounts
163     WHERE AccountID = p_from
164     FOR UPDATE;
165
166     IF v_balance < p_amount THEN
167         RAISE_APPLICATION_ERROR(-20001, 'Insufficient Funds');
168     END IF;
169
170     UPDATE Accounts
171     SET Balance = Balance - p_amount
172     WHERE AccountID = p_from;
173
174     UPDATE Accounts
175     SET Balance = Balance + p_amount
176     WHERE AccountID = p_to;
177
178     COMMIT;
179
180 EXCEPTION
181     WHEN OTHERS THEN
182         ROLLBACK;
183         DBMS_OUTPUT.PUT_LINE (SQLERRM);
184 END;

```

Output:

Procedure SAFETRANSFERFUNDS compiled

Elapsed: 00:00:00.020

- **Scenario 2:** Manage errors when updating employee salaries.
Question: Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

Code:

```

186
187 CREATE OR REPLACE PROCEDURE UpdateSalary (
188     p_empid NUMBER,
189     p_percent NUMBER
190 )
191 IS
192 BEGIN
193     UPDATE Employees
194     SET Salary = Salary + (Salary*p_percent/100)
195     WHERE EmployeeID = p_empid;
196
197     IF SQL%ROWCOUNT = 0 THEN
198         RAISE NO_DATA_FOUND;
199     END IF;
200
201     COMMIT;
202
203 EXCEPTION
204     WHEN NO_DATA_FOUND THEN
205         DBMS_OUTPUT.PUT_LINE('Employee ID does not exist.');
```

Output:

```
Procedure UPDATESALARY compiled  
Elapsed: 00:00:00.016
```

- **Scenario 3:** Ensure data integrity when adding a new customer.

Question: Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

Code:

```
213  
214 CREATE OR REPLACE PROCEDURE AddNewCustomer(  
215     p_id NUMBER,  
216     p_name VARCHAR2,  
217     p_dob DATE,  
218     p_balance NUMBER  
219 )  
220 IS  
221 BEGIN  
222     INSERT INTO Customers  
223     VALUES (p_id,p_name,p_dob,p_balance,SYSDATE,'N');  
224  
225     COMMIT;  
226  
227 EXCEPTION  
228     WHEN DUP_VAL_ON_INDEX THEN  
229         DBMS_OUTPUT.PUT_LINE('Customer already exists.');
```

Output:

```
Procedure ADDNEWCUSTOMER compiled  
Elapsed: 00:00:00.027
```

Exercise 3: Stored Procedures

- **Scenario 1:** The bank needs to process monthly interest for all savings accounts.

Question: Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Code:

```
235
236
237 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
238 IS
239 BEGIN
240     UPDATE Accounts
241     SET Balance = Balance + (Balance*0.01)
242     WHERE AccountType='Savings';
243
244     COMMIT;
245 END;
```

Output:

```
Procedure PROCESSMONTHLYINTEREST compiled

Elapsed: 00:00:00.014
```

- **Scenario 2:** The bank wants to implement a bonus scheme for employees based on their performance.

Question: Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Code:

```
249
250 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
251     p_department VARCHAR2,
252     p_bonus NUMBER
253 )
254 IS
255 BEGIN
256     UPDATE Employees
257     SET Salary = Salary + (Salary*p_bonus/100)
258     WHERE Department = p_department;
259
260     COMMIT;
261 END;
```

Output:

```
Procedure UPDATEEMPLOYEEBONUS compiled

Elapsed: 00:00:00.017
```

- **Scenario 3:** Customers should be able to transfer funds between their accounts.
Question: Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Code:

```
267 CREATE OR REPLACE PROCEDURE TransferFunds(  
268     p_from NUMBER,  
269     p_to NUMBER,  
270     p_amount NUMBER  
271 )  
272 IS  
273     v_balance NUMBER;  
274 BEGIN  
275  
276     SELECT Balance  
277     INTO v_balance  
278     FROM Accounts  
279     WHERE AccountID = p_from  
280     FOR UPDATE;  
281  
282     IF v_balance >= p_amount THEN  
283  
284         UPDATE Accounts  
285         SET Balance = Balance - p_amount  
286         WHERE AccountID = p_from;  
287  
288         UPDATE Accounts  
289         SET Balance = Balance + p_amount  
290         WHERE AccountID = p_to;  
291  
292         COMMIT;  
293  
294     ELSE  
295         DBMS_OUTPUT.PUT_LINE('Insufficient Balance');  
296     END IF;  
297  
298 END;
```

Output:

Procedure TRANSFERFUNDS compiled

Elapsed: 00:00:00.017

Exercise 4: Functions

- **Scenario 1:** Calculate the age of customers for eligibility checks.

Question: Write a function CalculateAge that takes a customer's date of birth as input and returns their age in years.

Code:

```
302 CREATE OR REPLACE FUNCTION CalculateAge(  
303     p_dob DATE  
304 )  
305 RETURN NUMBER  
306 IS  
307 BEGIN  
308     RETURN FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);  
309 END;
```


Output:

```
Function CALCULATEAGE compiled
```

```
Elapsed: 00:00:00.015
```

- **Scenario 2:** The bank needs to compute the monthly installment for a loan.
Question: Write a function CalculateMonthlyInstallment that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

Code:

```
314 CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(  
315     p_amount NUMBER,  
316     p_rate NUMBER,  
317     p_years NUMBER  
318 )  
319 RETURN NUMBER  
320 IS  
321 BEGIN  
322     RETURN (p_amount + (p_amount * p_rate * p_years / 100)) / (p_years * 12);  
323 END;
```

Output:

```
Function CALCULATEMONTHLYINSTALLMENT compiled
```

```
Elapsed: 00:00:00.013
```

- **Scenario 3:** Check if a customer has sufficient balance before making a transaction.
Question: Write a function HasSufficientBalance that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

Code:

```
327  
328 CREATE OR REPLACE FUNCTION HasSufficientBalance(  
329     p_account NUMBER,  
330     p_amount NUMBER  
331 )  
332 RETURN BOOLEAN  
333 IS  
334     v_balance NUMBER;  
335 BEGIN  
336  
337     SELECT Balance  
338     INTO v_balance  
339     FROM Accounts  
340     WHERE AccountID = p_account;  
341  
342     RETURN v_balance >= p_amount;  
343  
344 EXCEPTION  
345     WHEN NO_DATA_FOUND THEN  
346         RETURN FALSE;  
347 END;
```

Output:

```
Function HASSUFFICIENTBALANCE compiled
```

```
Elapsed: 00:00:00.013
```

Exercise 5: Triggers

- **Scenario 1:** Automatically update the last modified date when a customer's record is updated.

Question: Write a trigger **UpdateCustomerLastModified** that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

Code:

```
351 CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
352 BEFORE UPDATE
353 ON Customers
354 FOR EACH ROW
355 BEGIN
356     :NEW.LastModified := SYSDATE;
357 END;
```

Output:

```
Trigger UPDATECUSTOMERLASTMODIFIED compiled
```

```
Elapsed: 00:00:00.015
```

- **Scenario 2:** Maintain an audit log for all transactions.

Question: Write a trigger **LogTransaction** that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

Code:

```
362 CREATE OR REPLACE TRIGGER LogTransaction
363 AFTER INSERT
364 ON Transactions
365 FOR EACH ROW
366 BEGIN
367     INSERT INTO AuditLog
368     (TransactionID, LogDate, Action)
369     VALUES
370     (:NEW.TransactionID, SYSDATE, 'Transaction Inserted');
371 END;
```

Output:

```
Trigger LOGTRANSACTION compiled
```

```
Elapsed: 00:00:00.017
```

- **Scenario 3:** Enforce business rules on deposits and withdrawals.

Question: Write a trigger **CheckTransactionRules** that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

Code:

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT
ON Transactions
FOR EACH ROW

DECLARE
    v_balance NUMBER;

BEGIN

    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID=:NEW.AccountID;

    IF :NEW.TransactionType='Withdrawal'
        AND :NEW.Amount>v_balance THEN

        RAISE_APPLICATION_ERROR(-20002,'Insufficient Balance');

    END IF;

    IF :NEW.TransactionType='Deposit'
        AND :NEW.Amount<=0 THEN

        RAISE_APPLICATION_ERROR(-20003,'Deposit must be positive');

    END IF;

END;
```

Output:

Trigger CHECKTRANSACTIONRULES compiled

Elapsed: 00:00:00.020

Exercise 6: Cursors

- **Scenario 1:** Generate monthly statements for all customers.

Question: Write a PL/SQL block using an explicit cursor

GenerateMonthlyStatements that retrieves all transactions for the current month and prints a statement for each customer.

Code:

```
DECLARE
CURSOR c1 IS
SELECT *
FROM Transactions
WHERE TRUNC(TransactionDate, 'MM') = TRUNC(SYSDATE, 'MM');

rec c1%ROWTYPE;

BEGIN

OPEN c1;

LOOP

FETCH c1 INTO rec;

EXIT WHEN c1%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(
'Transaction ID: ' || rec.TransactionID ||
' Amount: ' || rec.Amount ||
' Type: ' || rec.TransactionType);

END LOOP;

CLOSE c1;

END;
```

Output:

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014

- **Scenario 2:** Apply annual fee to all accounts.

Question: Write a PL/SQL block using an explicit cursor **ApplyAnnualFee** that deducts an annual maintenance fee from the balance of all accounts.

Code:

```
439 DECLARE
440 CURSOR c1 IS
441 SELECT AccountID
442 FROM Accounts
443 FOR UPDATE;
444
445 v_fee NUMBER := 100;
446
447 BEGIN
448
449 FOR rec IN c1 LOOP
450
451 UPDATE Accounts
452 SET Balance = Balance - v_fee
453 WHERE CURRENT OF c1;
454
455 END LOOP;
456 COMMIT;
457 END;
```

Output:

```
PL/SQL procedure successfully completed.
```

```
Elapsed: 00:00:00.013
```

- **Scenario 3:** Update the interest rate for all loans based on a new policy.

Question: Write a PL/SQL block using an explicit cursor

UpdateLoanInterestRates that fetches all loans and updates their interest rates based on the new policy.

Code:

```
462 DECLARE
463
464 CURSOR c1 IS
465 SELECT LoanID, InterestRate
466 FROM Loans
467 FOR UPDATE;
468
469 BEGIN
470 |
471 FOR rec IN c1 LOOP
472
473 UPDATE Loans
474 SET InterestRate = InterestRate + 0.5
475 WHERE CURRENT OF c1;
476
477 END LOOP;
478
479 COMMIT;
480
481 END;
```

Output:

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010

Exercise 7: Packages

- **Scenario 1:** Group all customer-related procedures and functions into a package.

Question: Create a package **CustomerManagement** with procedures for adding a new customer, updating customer details, and a function to get customer balance.

Code: Package Specification

```
486 CREATE OR REPLACE PACKAGE CustomerManagement AS
487
488 PROCEDURE AddCustomer(
489     p_id NUMBER,
490     p_name VARCHAR2,
491     p_dob DATE,
492     p_balance NUMBER);
493
494 PROCEDURE UpdateCustomer(
495     p_id NUMBER,
496     p_balance NUMBER);
497
498 FUNCTION GetCustomerBalance(  
499     p_id NUMBER)  
500 RETURN NUMBER;  
501
502 END CustomerManagement;
```

Output:

Package CUSTOMERMANAGEMENT compiled

Elapsed: 00:00:00.014

Code: Package Body

```
504
505 CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
506 PROCEDURE AddCustomer(
507     p_id NUMBER,
508     p_name VARCHAR2,
509     p_dob DATE,
510     p_balance NUMBER)
511 IS
512 BEGIN
513     INSERT INTO Customers
514     VALUES (p_id,p_name,p_dob,p_balance,SYSDATE,'N');
515 END;
516 PROCEDURE UpdateCustomer(
517     p_id NUMBER,
518     p_balance NUMBER)
519 IS
520 BEGIN
521     UPDATE Customers
522     SET Balance=p_balance
523     WHERE CustomerID=p_id;
524 END;
525 FUNCTION GetCustomerBalance(
526     p_id NUMBER)
527 RETURN NUMBER
528 IS
529     v_balance NUMBER;
530 BEGIN
531     SELECT Balance
532     INTO v_balance
533     FROM Customers
534     WHERE CustomerID=p_id;
535
536     RETURN v_balance;
537 END;
538
539 END CustomerManagement;
```

Output:

Package Body CUSTOMERMANAGEMENT compiled

Elapsed: 00:00:00.014

- **Scenario 2:** Create a package to manage employee data.

Question: Write a package **EmployeeManagement** with procedures to hire new employees, update employee details, and a function to calculate annual salary.

Code:Package Specification

```
543 CREATE OR REPLACE PACKAGE EmployeeManagement AS
544
545     PROCEDURE HireEmployee(
546         p_id NUMBER,
547         p_name VARCHAR2,
548         p_position VARCHAR2,
549         p_salary NUMBER,
550         p_department VARCHAR2
551     );
552
553     PROCEDURE UpdateEmployee(
554         p_id NUMBER,
555         p_salary NUMBER
556     );
557
558     FUNCTION CalculateAnnualSalary(
559         p_id NUMBER
560     ) RETURN NUMBER;
561
562 END EmployeeManagement;
```

Output:

Package Body CUSTOMERMANAGEMENT compiled

Elapsed: 00:00:00.014

Code:Package Body

```
566 CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
567
568     PROCEDURE HireEmployee(
569         p_id NUMBER,
570         p_name VARCHAR2,
571         p_position VARCHAR2,
572         p_salary NUMBER,
573         p_department VARCHAR2
574     )
575     IS
576     BEGIN
577         INSERT INTO Employees
578         VALUES (
579             p_id,
580             p_name,
581             p_position,
582             p_salary,
583             p_department,
584             SYSDATE
585         );
586     END HireEmployee;
587
```

```
8     PROCEDURE UpdateEmployee (
9         p_id NUMBER,
10        p_salary NUMBER
11    )
12    IS
13    BEGIN
14        UPDATE Employees
15        SET Salary = p_salary
16        WHERE EmployeeID = p_id;
17    END UpdateEmployee;
18
19    FUNCTION CalculateAnnualSalary(
20        p_id NUMBER
21    )
22    RETURN NUMBER
23    IS
24        v_salary NUMBER;
25    BEGIN
26        SELECT Salary
27        INTO v_salary
28        FROM Employees
29        WHERE EmployeeID = p_id;
30
31        RETURN v_salary * 12;
32    END CalculateAnnualSalary;
33
34 END EmployeeManagement;
```

Output:

Package Body CUSTOMERMANAGEMENT compiled

Elapsed: 00:00:00.014

- **Scenario 3:** Group all account-related operations into a package.

Question: Create a package **AccountOperations** with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

Code: Package Specification

```

617 CREATE OR REPLACE PACKAGE AccountOperations AS
618
619     PROCEDURE OpenAccount(
620         p_accountid NUMBER,
621         p_customerid NUMBER,
622         p_type VARCHAR2,
623         p_balance NUMBER
624     );
625
626     PROCEDURE CloseAccount(
627         p_accountid NUMBER
628     );
629
630     FUNCTION GetTotalBalance(
631         p_customerid NUMBER
632     ) RETURN NUMBER;
633
634 END AccountOperations;

```

Output:

Package Body CUSTOMERMANAGEMENT compiled

Elapsed: 00:00:00.014

Code: Package Body:

```

640 CREATE OR REPLACE PACKAGE BODY AccountOperations AS
641
642     PROCEDURE OpenAccount(
643         p_accountid NUMBER,
644         p_customerid NUMBER,
645         p_type VARCHAR2,
646         p_balance NUMBER
647     )
648     IS
649     BEGIN
650         INSERT INTO Accounts
651         VALUES(
652             p_accountid,
653             p_customerid,
654             p_type,
655             p_balance,
656             SYSDATE
657         );
658     END OpenAccount;
659
660     PROCEDURE CloseAccount(
661         p_accountid NUMBER
662     )
663     IS
664     BEGIN
665         DELETE FROM Accounts
666         WHERE AccountID = p_accountid;
667     END CloseAccount;
668

```

```

668
669     FUNCTION GetTotalBalance(
670         p_customerid NUMBER
671     )
672     RETURN NUMBER
673     IS
674         v_total NUMBER;
675     BEGIN
676         SELECT NVL(SUM(Balance),0)
677         INTO v_total
678         FROM Accounts
679         WHERE CustomerID = p_customerid;
680
681         RETURN v_total;
682     END GetTotalBalance;
683
684 END AccountOperations;

```

Output:

```
Package Body CUSTOMERMANAGEMENT compiled
```

```
Elapsed: 00:00:00.014
```